

Informe desafío 1

Sweet Crush

Santiago Ortiz
Alejandro Serna

Universidad de Antioquia
Septiembre 2026

1 Análisis del problema y consideraciones para la alternativa de solución propuesta

Este juego nos desafía a ser estrictos con el manejo de los bits y la memoria dinámica, empezando por eso, decidimos crear un el tablero donde viven todas las fichas de manera dinámica, la primera generación de este se da cuando el usuario define por primera vez la cantidad de filas y columnas pero, esta área puede variar con las opciones de agregar/eliminar filas o columnas.

Con esto en mente definimos las dos funciones principales que manejan los bits (fichas) dentro del tablero `setFicha` y `getFicha` que las explicamos más adelante, algunas consideraciones que tomamos fueron las siguientes: no se puede crear un tablero dinámico excesivamente grande, o sea, con espacio de más al principio, en el arreglo del tablero, las fichas deben ser contiguas, no pueden quedar espacios vacíos entre los bits, también al eliminar una ficha o una combinación válida las fichas de arriba deben caer, pudiendo crear algunas combinaciones llamadas "cascadas", que se deben tomar en cuenta también para considerar el puntaje del jugador.

2 Esquema donde describa las tareas que usted definió en el desarrollo de los algoritmos

2.1 Manipular tablero

Esta es la tarea principal, donde por medio de dos funciones básicas podemos ingresar y extraer las fichas de manera organizada.

2.2 Visualización del tablero

Esta parte nos permite interactuar mediante ciertas opciones del usuario con el juego como tal. Además de generar primero un tablero completamente aleatorio y desordenado.

2.3 Detección de fichas

Esta parte se encarga de verificar si hubo combinaciones válidas, para marcarlas y luego modificar su valor y proceder a sumar puntos al jugador.

2.4 Reorganizar el tablero visualmente

Esta sección se encarga de bajar las fichas de arriba hacia abajo, además de rellenar los huecos faltantes con fichas aleatorias.

2.5 Manejo memoria dinámica

Esta es la parte más relevante, consta de ciertas funciones que dependiendo de algunas variables como puede ser el espacio total ocupado por las fichas, la dimensión del tablero, etc. puede cambiar físicamente la memoria empleada en el programa.

3 Algoritmos implementados

3.1 En función: setFicha

Esta función mediante su algoritmo lo que hace principalmente es mediante un índice que es la ubicación del primer bit de la ficha, una máscara con el número 7 (en decimal nos da 3 bits en 1) podemos ubicar sin modificar el resto del byte sin tocar lo que está alrededor e ingresar una ficha (3 bits) en la posición que necesitamos, es decir, se limpia el byte de cierta manera para ubicar una nueva ficha pero sin perder información, modifica el arreglo principal del tablero, ubica las fichas de izquierda a derecha, es decir, al hallar donde empieza el primer bit, a 5 le restamos esa cantidad para que esos bits siempre nos queden en las posiciones 5, 6 y 7 para añadirlos en un byte, pero si por ejemplo esa ficha no cabe dentro del byte actual, se debe partir en dos bytes, sabemos cuántos bits deben ir en el byte siguiente mediante otra operación matemática que nos dará 0 o 1 o 2 bits, luego repetimos el proceso para agregarlos en las posiciones correspondientes.

3.2 En función: getFicha

Esta función se basa en un algoritmo que hace básicamente lo contrario a setFicha, igualmente usando una máscara de 3 bits continuos en 1, o sea, en decimal el 7, se usa esta máscara para extraer los bits que necesitamos sabiendo antes si en el bit en que inicia está totalmente dentro de un byte, sabemos esto mediante un índice y una operación matemática. Luego la función se encarga de recuperar los bits de una ficha y darnos su valor decimal, esto para poder mostrar el tablero que verá el usuario, es básicamente una función consultora sobre el arreglo principal.

3.3 En función: Detectar horizontal

El algoritmo de esta función es básicamente una búsqueda lineal en una fila y vamos recorriendo las columnas de esta, dejamos fijo siempre la primera columna de cada fila para comparar, por eso invocamos a getFicha pero con un cero fijo en las columnas; esto lo hacemos hasta iterar en todas las filas actuales del tablero y, como vamos comparando por ficha, cuando la ficha del actual es igual a la que le sigue vamos sumando el número de fichas para saber si hay una combinación válida, luego preguntamos si ese contador es mayor o igual a 3 y, volvemos a iterar en ese índice donde se encuentran las 3 o más fichas iguales marcándolas, este marcado es sencillo, en un arreglo dinámico de bool con todos los valores en falso al inicio, al iterar en ese índice vamos cambiando los valores de FALSO a VERDADERO en esa posición.

3.4 En función: Detectar vertical

El algoritmo de esta función es muy similar al de detectar combinaciones en horizontales (búsqueda lineal), solo que ahora verticales, fijamos todos los caracteres de la fila cero para hacer la comparación, vamos bajando de fila y hacemos el mismo proceso, si las fichas contiguas son iguales se suma 1 al contador, si son diferentes se verifica si el contador es mayor o igual a 3 para comprobar si hubo una combinación.

4 Problemas de desarrollo que afrontó

Al momento de desarrollar la solución del desafío pasamos por varios inconvenientes, por ejemplo, la primera versión de la función setFicha que agrega la terna de bits, lo hacía de izquierda a derecha, esto nos generaba cierta complejidad al agregar más datos al arreglo, por lo que al final se decidió cambiarla.

En general también la depuración se torna compleja ya que depende mucho de factores aleatorios, por ejemplo, la generación de fichas o filas o columnas completas, ya que en medio de este proceso puede haber combinaciones válidas, lo que genera más confusión aún. También se nos hizo complejo en general el método de detección de combinaciones válidas, ya que no sabíamos si era mejor ir sobrescribiendo el tablero o solo marcarlo de cierta forma para saber dónde

estaban esas combinaciones, al final decidimos la segunda opción de ir marcando dichas posiciones de las combinaciones válidas, principalmente ya que no nos daba cuando una ficha hacía parte de una combinación horizontal y vertical; el anterior algoritmo daba un resultado erróneo; es básicamente una búsqueda lineal que marca esas posiciones de las fichas que forman una combinación válida. También nos costó al principio visualizar cuál era la mejor estructura para almacenar los bits, ya que veíamos ventajas en una y en otras no de manera analítica,

5 Evolución de la solución y consideraciones para tener en cuenta en la implementación

Primero empezamos con un enfoque de un arreglo lineal para guardar las fichas del tablero, lo que hicimos fue enfocarnos principalmente en dos funciones principales importantes para el desarrollo de este, que son `setFicha` y `getFicha`, mediante estas funciones podemos escribir y obtener los bits de las fichas para manipularlas, luego pensamos en mostrar un tablero inicial completamente aleatorio, esto haciendo uso de las dos funciones, una vez el tablero relleno pasamos a generar las funciones detectoras de combinaciones que son el alma básica del juego, teniendo en cuenta el uso de la memoria dinámica y el manejo a nivel de bits de manera eficiente y organizada, luego pasamos a las funciones que le permiten interactuar al jugador, como son las de quitar fichas o quitar/agregar filas y columnas.

Una vez el esqueleto completado se pasó a crear una interfaz un poco más entendible para el usuario donde se puede visualizar de buena manera el número de cada fila y columna del tablero así como sus fichas altamente diferenciables.